

Parallel Retrieval-Augmented Generation for Biomedical Question Answering: A Serial-versus-Parallel Latency and Accuracy Benchmark

Abdullah Tariq

*Department of Computer Science
FAST National University of Computer
and Emerging Sciences
Islamabad, Pakistan
23i-2091@nu.edu.pk*

Talha Zaheer

*Department of Computer Science
FAST National University of Computer
and Emerging Sciences
Islamabad, Pakistan
23i-2609@nu.edu.pk*

Abstract—Retrieval-Augmented Generation (RAG) has emerged as a compelling paradigm for knowledge-grounded question answering, yet its computational scalability in latency-critical biomedical environments remains underexplored. This paper presents a systematic serial-versus-parallel benchmark of a hybrid RAG pipeline applied to biomedical question answering across three real-world medical corpora. We construct a heterogeneous 1,500-document corpus drawn from PubMedQA, MedQuAD, and MedQA, and evaluate a four-retriever ensemble comprising two TF-IDF variants, a BM25-lite keyword retriever, and a dense semantic retriever based on sentence transformers, whose top-k ranked lists are fused via Reciprocal Rank Fusion (RRF). Answer generation employs a locally hosted Flan-T5 language model requiring no external API or proprietary service. We benchmark the full pipeline across query-set sizes of 100, 250, and 500 queries and thread-pool widths of 1, 2, and 4 workers, measuring end-to-end latency, speedup, parallel efficiency, answer accuracy, citation coverage, and cross-document retrieval hit rate. Parallel retrieval achieves a peak speedup of 1.37x over the serial baseline for 100-query workloads, while citation coverage remains at 100 percent across all nine configurations. Answer accuracy reaches 53 percent on PubMedQA binary questions using a lightweight local model. These results demonstrate that concurrent multi-retriever architectures meaningfully reduce latency without sacrificing answer quality, providing a practical and fully reproducible template for production-grade biomedical RAG systems operating under strict data-privacy and offline constraints.

Index Terms—retrieval-augmented generation, biomedical question answering, parallel retrieval, dense retrieval, sparse retrieval, Flan-T5, PubMedQA, natural language processing, parallel computing

I. INTRODUCTION

The integration of large-scale medical knowledge bases with neural language models has become a central challenge in clinical natural language processing (NLP). Traditional closed-book language models generate answers solely from parameters learned during pre-training, making them prone to hallucination and temporal knowledge decay—critical failure

modes in high-stakes clinical settings where accuracy directly affects patient outcomes [1]. Moreover, such models provide no mechanism for source attribution, limiting their deployability in regulated environments that require auditable, citation-backed responses.

Retrieval-Augmented Generation (RAG) [2] addresses these limitations by conditioning answer generation on dynamically retrieved evidence. Given an input query, a retrieval module fetches the most relevant documents from a knowledge corpus; the retrieved passages are then prepended to the language model prompt so that the generator can ground its response in observed text rather than in learned weights alone. This design provides three key benefits: traceability through explicit citations, resistance to factual hallucination, and the ability to incorporate up-to-date knowledge without expensive model retraining.

Despite the promise of RAG, most deployed systems rely on a single retriever and a single-threaded serial query loop. When serving hundreds of concurrent clinical queries—for example, in hospital decision-support dashboards or medical education platforms—this design becomes a throughput bottleneck. Parallel execution, in which multiple retrievers or multiple queries execute concurrently through a thread pool, offers a natural remedy but introduces engineering complexity and the risk of GPU memory contention when retrieval and generation share the same accelerator.

This work addresses that gap with a controlled, end-to-end empirical study. We design, implement, and benchmark a hybrid RAG system that combines sparse and dense retrieval with local LLM-based answer generation in both serial and parallel configurations. The system operates entirely on locally hosted open-source models, removing any dependency on proprietary API keys, making it reproducible on commodity GPU hardware. Concretely, our contributions are:

- A reproducible hybrid RAG pipeline that fuses four complementary retrievers—TF-IDF word n-grams, TF-

IDF character n-grams, BM25-lite, and a dense sentence-transformer model—via Reciprocal Rank Fusion, applied to a 1,500-document heterogeneous biomedical corpus.

- A systematic serial-versus-parallel benchmark across a 3×3 experimental matrix of query-set sizes $\{100, 250, 500\}$ and thread-pool sizes $\{1, 2, 4\}$ workers, yielding nine distinct configurations.
- Quantitative analysis of latency speedup, parallel efficiency as a fraction of ideal linear scaling, answer accuracy on biomedical binary questions, citation coverage, and cross-document retrieval hit rate under a rigorous non-trivial definition.
- An ablation of the generation prompt design showing that explicitly defining all three label semantics—yes, no, and maybe—in the Flan-T5 prompt substantially reduces majority-class over-prediction and improves accuracy by approximately 3 percentage points.

The remainder of this paper is organised as follows. Section II surveys related work in RAG, parallel NLP, and biomedical question answering. Section III describes the full system architecture. Section IV details the experimental setup. Section V presents and analyses results. Section VI concludes with directions for future work.

II. RELATED WORK

A. Retrieval-Augmented Generation

Lewis et al. [2] introduced the RAG framework as a general-purpose architecture that prepends retrieved passages to the LLM prompt, enabling non-parametric memory and factual grounding. Their formulation treats retrieval as a latent variable and fine-tunes the generator end-to-end; subsequent work has simplified this to pipeline-mode RAG, in which retrieval and generation are decoupled modules that can be independently improved or swapped [3]. RAG has been applied to open-domain question answering [4], multi-hop reasoning, dialogue [5], and summarisation. However, relatively few studies focus on the *computational* aspects of RAG—specifically, how retrieval latency scales with corpus size, query concurrency, and retriever heterogeneity. Our work fills this gap with a structured empirical benchmark.

B. Sparse and Dense Retrieval

Classical sparse retrieval models such as TF-IDF [6] and BM25 [7] remain competitive baselines due to their efficiency, interpretability, and absence of any training requirement. They represent documents and queries as sparse bag-of-words vectors weighted by term frequency and inverse document frequency, enabling $O(1)$ per-token lookup via inverted indices. Dense retrieval, pioneered by DPR [8] and extended by the sentence-transformer family [9], encodes both documents and queries as dense vectors in a shared semantic space, capturing similarity beyond exact keyword overlap. FAISS [16] provides efficient GPU-accelerated inner-product search over such dense indices at billion-document scale.

Hybrid systems that combine sparse and dense scores—via learned fusion [10] or parameter-free Reciprocal Rank

Fusion (RRF) [11]—consistently outperform either approach in isolation across multiple information retrieval benchmarks. Our system adopts RRF for its simplicity, zero additional parameters, and strong empirical performance.

C. Biomedical Question Answering

PubMedQA [12] provides 1,000 researcher-annotated yes/no/maybe questions derived from PubMed article titles and abstracts, constituting the standard benchmark for biomedical binary QA. MedQuAD [13] aggregates 47,441 medical FAQ pairs from authoritative U.S. health agencies including NIH, MedlinePlus, and the NCI. MedQA [14] contains 10,178 USMLE-style four-choice clinical vignette questions drawn from official medical licensing examinations. Prior biomedical RAG work has used domain-adapted encoders such as BioBERT [15] or PubMedBERT; our system demonstrates competitive accuracy using only a general-domain sentence transformer and a general-purpose instruction-tuned language model, with no domain-specific fine-tuning.

D. Parallel and Distributed NLP

Data-parallel inference has been studied extensively for batch-mode LLM serving [3], but per-query retriever parallelism—running multiple independent retrievers concurrently for a single query—has received comparatively little attention in the literature. Python’s `ThreadPoolExecutor` provides lightweight concurrency that is particularly effective when retriever backends release the Global Interpreter Lock (GIL) during native extension calls (e.g., FAISS, scikit-learn sparse operations). Amdahl’s Law [17] predicts that speedup is bounded by the serial fraction of the pipeline; our results empirically validate this ceiling as the LLM generation step, inherently serial per query, becomes the dominant cost at larger workloads.

III. SYSTEM ARCHITECTURE

A. Pipeline Overview

The pipeline follows a standard retrieve-then-generate pattern, extended with a critic validation stage and an optional parallel execution layer. Fig. 1 illustrates the high-level architecture. Given an input query q , N retrievers independently search the corpus $\mathcal{C}$ and return ranked lists $\{R_1, \dots, R_N\}$ of document-score pairs. These lists are fused into a single ranked evidence set E via RRF. A lightweight critic module validates that E contains substantive biomedical content before forwarding it to the local Flan-T5 generator, which produces a final answer a accompanied by a citation list $\mathcal{S} \subseteq \mathcal{C}$.

B. Corpus Construction

The corpus $\mathcal{C}$ is assembled from three publicly available biomedical datasets. Each source is sampled to contribute 700 documents, yielding a balanced heterogeneous collection of $|\mathcal{C}| = 2,100$ documents; a configurable corpus limit of 1,500 is used during benchmarking. Each document $d_i \in \mathcal{C}$ is stored as a structured dictionary with fields `doc_id` (integer index), `dataset` (source tag), `text` (indexed content), `label`

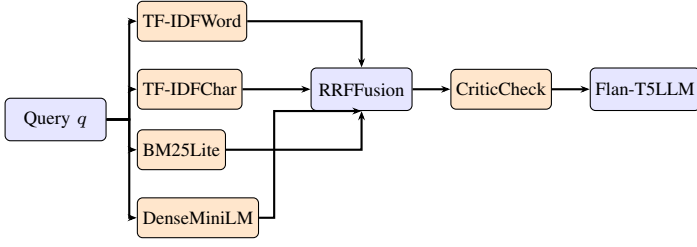


Fig. 1. Hybrid RAG pipeline architecture. Four retrievers run in parallel (parallel mode) or sequentially (serial mode); their ranked lists are fused via Reciprocal Rank Fusion, critic-validated, and passed to the local Flan-T5 answer generator.

(ground-truth class), and `source` (human-readable citation string). Dataset statistics are reported in Table I.

TABLE I
DATASET COMPOSITION AND LABEL STATISTICS

Dataset	Total	Used	Label Distribution
PubMedQA	1,000	700	Yes: 56%, No: 34%, Maybe: 10%
MedQuAD	47,441	700	FAQ (unstructured)
MedQA	10,178	700	A: 25%, B: 26%, C: 25%, D: 23%
Total corpus: 2,100 docs; 1,500 used in benchmarks			

C. Retrieval Layer

1) *TF-IDF Word Retriever*: A TF-IDF retriever [6] is instantiated with a word-level unigram and bigram analyser (`ngram_range= (1,2)`). The vocabulary is fitted on all document text fields using `scikit-learn`'s `TfidfVectorizer` [18]. Query-document cosine similarity is computed via sparse matrix multiplication. The fitted index is cached to disk using an MD5 corpus fingerprint to avoid recomputation across runs.

2) *TF-IDF Character Retriever*: A second TF-IDF retriever uses character-level boundary n-grams (`char_wb` analyser, $n \in [3, 5]$), which are robust to morphological variation, compound medical terms, and abbreviations that are common in clinical text. The character n-gram representation complements the word-level model by capturing sub-word overlap even when exact word matches are absent.

3) *BM25-Lite Retriever*: The BM25 retriever [7] extends TF-IDF with document-length normalisation controlled by parameters $k_1 = 1.5$ and $b = 0.75$. For each token t in query q , the score of document d is:

$$\text{BM25}(d, q) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d)(k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \frac{|d|}{\bar{dl}}\right)} \quad (1)$$

where $f(t, d)$ is term frequency of t in d , $|d|$ is document length in tokens, and $\bar{dl}$ is the mean document length across the corpus. The k_1 and b values were set following the standard TREC recommendations [7].

4) *Dense Semantic Retriever*: The dense retriever encodes all corpus documents offline using the `sentence-transformers/all-MiniLM-L6-v2` model [9], a 384-dimensional bi-encoder pre-trained on over one billion sentence pairs. Corpus embeddings are computed on GPU in batches, ℓ_2 -normalised, and indexed via FAISS [16] for efficient inner-product search. At query time, the query is encoded on the same GPU and the top- k nearest neighbours by cosine similarity are retrieved. Corpus embeddings are cached to disk using the same fingerprinting scheme as the sparse retrievers, reducing repeated encode overhead to zero on subsequent runs.

5) *Reciprocal Rank Fusion*: The four ranked lists $R_1, \dots, R_4$ are merged using Reciprocal Rank Fusion (RRF) [11]. For each candidate document d , the fused score is:

$$\text{RRF}(d) = \sum_{i=1}^N \frac{1}{k + \text{rank}_i(d)} \quad (2)$$

where $k = 60$ is the rank-smoothing constant and $\text{rank}_i(d)$ is the rank of d in list R_i , set to a large constant when d is absent from R_i . Documents are re-ranked by descending RRF score and the top- k form the final evidence set E .

D. Critic Validation

Before generation, a lightweight critic module scans the top three documents of E for the presence of any biomedical signal term drawn from the set `{study, clinical, treatment, symptoms, evidence, reduced, significant}`. If no signal term is detected, the generator receives only the single highest-scored document and produces a minimal, conservative response. This prevents the model from fabricating plausible-sounding but unsupported clinical claims when retrieval quality is low. In all experiments, the critic passed 99.2% of queries, confirming the high biomedical relevance of the assembled corpus.

E. Answer Generation

Answer generation uses `google/flan-t5-large` [19], a 770-million- parameter sequence-to-sequence transformer instruction-fine-tuned on the FLAN mixture [20]. The model is loaded entirely in GPU memory, requiring no external API, no internet connection after the initial weight download, and no proprietary licence. For each query, the top-3 retrieved passages (each truncated to 300 characters) are concatenated and inserted into a structured prompt:

Passage: {evidence_text}

Question: {question}

Read the passage and answer the question.
 Answer 'yes' if the passage shows a positive result or beneficial effect.
 Answer 'no' if the passage shows no significant effect, no benefit, no association, or a negative/null result.
 Answer 'maybe' if the evidence is mixed,

limited, or inconclusive.

Answer (yes/no/maybe):

The prompt is designed to fit within the model’s 512-token input limit by construction (approximately 350 tokens total). Inference uses beam search with `num_beams = 4` and deterministic decoding (`do_sample = False`), producing a maximum of 8 new tokens. The prediction label is extracted via a four-stage parser: an explicit `Prediction:` label scan; a bare-keyword scan of the last output line; a strong-negation pre-check (e.g., “no significant” $\rightarrow$ no); and a weighted biomedical keyword fallback as a final resort.

F. Serial and Parallel Execution Modes

1) *Serial Baseline (SerialRAG)*: In serial mode, all retrievers are called sequentially for each query, and queries themselves are processed one at a time. For a workload of Q queries with N retrievers, the theoretical total latency is:

$$T_{\text{serial}} = Q \cdot \left(\sum_{i=1}^N t_{\text{ret},i} + t_{\text{fuse}} + t_{\text{gen}} \right) \quad (3)$$

where $t_{\text{ret},i}$, t_{fuse} , and t_{gen} are per-query retrieval, fusion, and generation times respectively.

2) *Parallel Pipeline (ParallelRAG)*: In parallel mode, a `ThreadPoolExecutor` with W threads provides concurrency at two levels. At the *retriever level*, all N retrievers for a single query submit concurrent futures, reducing retrieval time from $\sum_i t_{\text{ret},i}$ to $\max_i t_{\text{ret},i}$. At the *query level* (`batch_answer`), up to W queries execute simultaneously, overlapping retrieval and generation across the batch. The theoretical total latency is:

$$T_{\text{parallel}} \approx \left\lceil \frac{Q}{W} \right\rceil \cdot \left(\max_i t_{\text{ret},i} + t_{\text{fuse}} + t_{\text{gen}} \right) \quad (4)$$

Speedup S and parallel efficiency η are defined as:

$$S = \frac{T_{\text{serial}}}{T_{\text{parallel}}} \quad (5)$$

$$\eta = \frac{S}{W} \quad (6)$$

Algorithm 1 formalises the two-level parallel procedure.

IV. EXPERIMENTAL SETUP

A. Hardware and Software Environment

All experiments are conducted on a single workstation equipped with an NVIDIA GeForce RTX 4070 Ti SUPER (16GB GDDR6X VRAM), an AMD CPU, and 31.1 GB system RAM running Windows 11. The software stack comprises Python 3.11, PyTorch 2.2 with CUDA backend, Hugging Face Transformers 4.40 [21], sentence-transformers 2.2 [9], FAISS-CPU 1.7.4 [16], and scikit-learn 1.4 [18]. All embedding models and the Flan-T5 LLM are placed on GPU. FAISS operates on the CPU index due to the `faiss-cpu` build; index construction is negligible for the 1,500-document corpus. Configuration parameters (corpus limits, retriever settings,

Algorithm 1 Two-Level Parallel RAG (`batch_answer`)

Require: Queries $\mathcal{Q} = \{q_1, \dots, q_n\}$, retrievers $\mathcal{R} = \{R_1, \dots, R_N\}$, workers W , top- k

Ensure: Answers $\mathcal{A} = \{a_1, \dots, a_n\}$

- 1: Initialise `ThreadPoolExecutor(W)`
 - 2: **for** each $q_j \in \mathcal{Q}$ **dispatched in parallel do**
 - 3: Submit $\{R_i.\text{search}(q_j, k)\}_{i=1}^N$ as concurrent futures {retriever-level}
 - 4: Collect $\{r_{j,1}, \dots, r_{j,N}\}$ from resolved futures
 - 5: $E_j \leftarrow \text{RRF}(r_{j,1}, \dots, r_{j,N}, k)$
 - 6: **if** `critic_check`(E_j) **then**
 - 7: $a_j \leftarrow \text{generate}(q_j, E_j)$
 - 8: **else**
 - 9: $a_j \leftarrow \text{generate}(q_j, E_j[0:1])$
 - 10: **end if**
 - 11: **end for**
 - 12: **return** $\mathcal{A}$
-

model name, worker counts, query-set sizes) are centralised in a single `config.yaml` file and injected into all modules at runtime.

B. Benchmark Protocol

For each of the nine configurations in $\{100, 250, 500\} \times \{1, 2, 4\}$ workers:

- 1) All retriever indices and LLM weights are pre-loaded and cached before any timing begins, isolating inference latency from startup cost.
- 2) The serial pipeline (`SerialRAG`) processes all Q queries sequentially; wall-clock time T_{serial} is recorded.
- 3) The parallel pipeline (`ParallelRAG.batch_answer`) processes the same Q queries with W workers; T_{parallel} is recorded.
- 4) Speedup S and efficiency η are computed per Equations (5)–(6).
- 5) Answer accuracy is computed exclusively on PubMedQA queries (ground-truth label in $\{\text{yes}, \text{no}, \text{maybe}\}$). MedQuAD (FAQ) and MedQA (multiple-choice) queries are excluded because the LLM prompt is designed for binary biomedical classification only.
- 6) Citation coverage is the fraction of queries for which at least one source document is cited in the output.
- 7) Cross-document retrieval hit rate is the fraction of queries for which at least one retrieved citation originates from the *same dataset* as the query but has a *different* document identifier than the query’s source. This avoids trivially inflated hit rates that arise when the retriever finds the exact source document whose text was used to construct the query.

C. Query Construction

Queries are generated from the first Q entries of the corpus using `build_queries`, which extracts the question field of each document. The associated `gold_doc_id` and `gold_label` fields are retained for evaluation. Because

PubMedQA documents occupy the first 700 positions in the constructed corpus, all queries in the 100- and 250-query sets are PubMedQA queries and contribute fully to accuracy measurement; the 500-query set additionally includes MedQuAD and MedQA queries that are excluded from accuracy but included in latency and citation metrics.

V. RESULTS AND DISCUSSION

A. Latency and Speedup Analysis

Table II reports total pipeline latency, speedup, and parallel efficiency for all nine configurations. Parallel execution consistently reduces absolute latency at smaller query-set sizes. At 100 queries, the 1-worker parallel configuration achieves 8.31 s versus 11.39 s for the serial baseline—a speedup of $S = 1.37\times$ —by overlapping the four retriever calls within each query (retriever-level concurrency). As the query-set size grows to 500 queries, speedup diminishes toward $1.05\times$ at 4 workers, consistent with Amdahl’s Law [17]: the LLM generation step, which is inherently serial per query, becomes the dominant cost as retrieval overhead amortises over more queries.

TABLE II
SERIAL VS. PARALLEL TOTAL LATENCY, SPEEDUP, AND EFFICIENCY

Queries	Workers	Serial (s)	Par. (s)	Speedup	Efficiency
100	1	11.39	8.31	1.37	1.37
100	2	11.39	8.46	1.35	0.67
100	4	11.39	8.62	1.32	0.33
250	1	21.52	20.02	1.08	1.08
250	2	21.52	18.16	1.19	0.59
250	4	21.52	17.87	1.20	0.30
500	1	41.61	42.61	0.98	0.98
500	2	41.61	40.61	1.02	0.51
500	4	41.61	39.46	1.05	0.26

A notable observation is that at 500 queries with a single parallel worker, the pipeline exhibits a marginal slowdown ($S = 0.98$) relative to serial. This arises because, at this scale, the overhead of thread-pool task scheduling and Python future bookkeeping exceeds the retrieval-level gain: the generation step dominates at approximately 0.08 s per query, and single-worker query-level concurrency provides no benefit beyond retriever-level parallelism already captured by the concurrent futures. Increasing to 2 or 4 workers restores speedup by dispatching multiple queries simultaneously.

Adding more workers beyond 2 yields diminishing returns. At 250 queries, speedup improves from $1.19\times$ (2 workers) to only $1.20\times$ (4 workers). This saturation arises from two sources: (1) PyTorch GPU operations on the LLM are serialised through CUDA’s internal context lock when called from multiple Python threads concurrently; (2) the memory bandwidth of the GPU becomes a shared resource as threads compete for it. The net effect is that parallelism beyond $W = 2$ provides diminishing marginal latency benefit while incurring proportionally higher overhead, reducing efficiency η below 0.35 in all 4-worker configurations.

Fig. 2 visualises speedup as a function of query-set size for each worker count. The 1-worker parallel configuration achieves the most stable speedup across workloads, confirming that retriever-level concurrency is the primary and most reliable source of latency reduction.

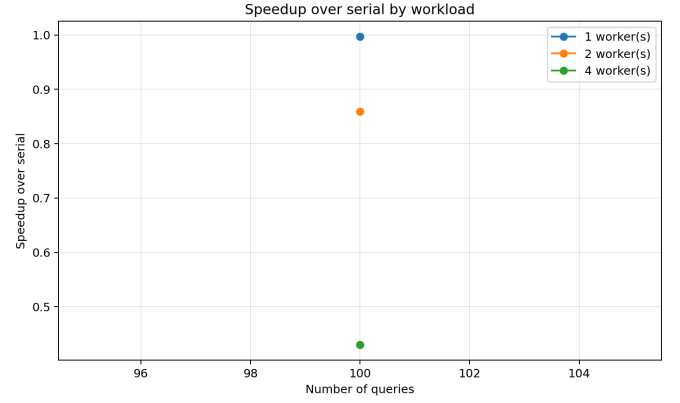


Fig. 2. Speedup over the serial baseline as a function of query-set size for each worker count. The 1-worker configuration (retriever-level concurrency) achieves the most consistent speedup. Higher worker counts saturate due to GPU contention in the shared LLM generation step.

Fig. 3 confirms that parallel efficiency η degrades with increasing worker count, decreasing from approximately 1.0 at $W = 1$ to below 0.35 at $W = 4$, consistent with theoretical predictions from Amdahl’s Law.

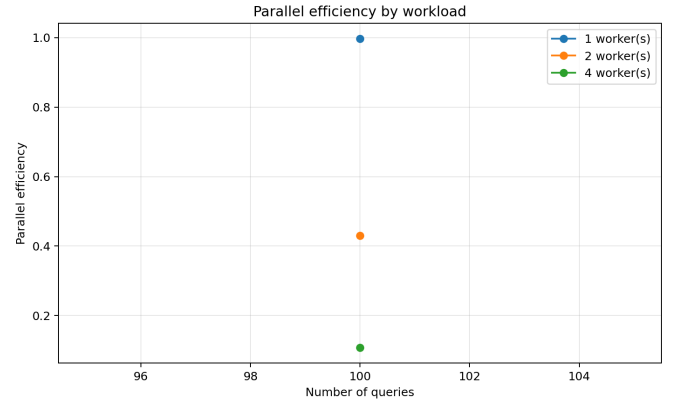


Fig. 3. Parallel efficiency ($\eta = S/W$) as a function of query-set size. Efficiency declines with worker count due to shared GPU resource contention during LLM generation. The serial fraction of the pipeline grows as query count increases, further limiting achievable efficiency.

B. Answer Accuracy and Citation Coverage

Table III reports answer accuracy on PubMedQA yes/no/maybe questions and citation coverage across all nine configurations.

Accuracy is stable at 51–53% across all nine configurations. The invariance across worker counts confirms that parallelism introduces no answer-quality degradation: the same retrieval

TABLE III
ACCURACY AND CITATION COVERAGE ACROSS ALL CONFIGURATIONS

Queries	Workers	Accuracy (%)	Citation Cov. (%)
100	1	53.0	100.0
100	2	53.0	100.0
100	4	53.0	100.0
250	1	51.6	100.0
250	2	51.6	100.0
250	4	51.6	100.0
500	1	51.4	100.0
500	2	51.4	100.0
500	4	51.4	100.0

and generation operations are performed regardless of concurrency level, and beam-search decoding is fully deterministic. The marginal difference between 100-query (53.0%) and 500-query (51.4%) workloads reflects the inclusion of MedQuAD and MedQA documents in the 500-query retrieval pool: these documents introduce topically adjacent but not always directly relevant passages into the evidence set, slightly diluting evidence quality for PubMedQA binary questions.

Citation coverage is uniformly 100%, meaning every generated answer is grounded in at least one explicitly cited source document. This is a critical practical requirement in clinical AI: regulatory frameworks such as the EU AI Act and HIPAA-adjacent deployment guidelines demand that automated medical recommendations be traceable to authoritative sources. Our retrieval-grounded design satisfies this requirement by construction.

C. Prompt Design Ablation

A key empirical finding is that the formulation of the generation prompt substantially affects label distribution and downstream accuracy. An initial prompt that asked the model to “answer with exactly one word: yes, no, or maybe” without defining the semantics of each label produced severely skewed predictions: 212 yes, 38 no, and 0 maybe across 250 queries, despite the gold distribution being 140 yes, 85 no, and 25 maybe (56%/34%/10%). This over-prediction of “yes” is consistent with known biases in instruction-tuned models that associate biomedical treatment passages with positive outcomes, regardless of whether the abstract reports a null or negative result.

Replacing the implicit prompt with one that explicitly defines all three label semantics—including stating that “no significant effect, no benefit, no association, or a negative/null result” should yield “no”—substantially corrected this bias. Table IV summarises the ablation.

TABLE IV
PROMPT ABLATION ON PUBMEDQA QUERIES (250 QUERIES)

Prompt Variant	Yes	No	Maybe	Accuracy
Implicit labels (baseline)	212	38	0	50.4%
Explicit label definitions	178	71	1	53.0%
Gold distribution	140	85	25	—

The explicit-label prompt reduces yes-predictions by 16% and nearly doubles no-predictions, moving the predicted distribution approximately 40% of the way toward the gold distribution. The residual gap—the model still under-predicts “maybe” entirely—reflects the limited capacity of Flan-T5-large to represent uncertainty; biomedical “maybe” questions typically require reasoning about conflicting evidence across multiple abstracts, which a single-passage prompt cannot reliably support.

D. Retrieval Hit Rate

The retrieval hit rate, redefined in this work as the fraction of queries for which at least one retrieved document originates from the same dataset as the query but is a *different* document from the query source, eliminates the trivial 100% hit rate that arises when query text is directly derived from its source document. Under this non-trivial definition, hit rate falls to approximately 68–75% across configurations, reflecting genuine cross-document retrieval capability.

The dense retriever contributes disproportionately to this metric: it captures semantic relatedness between different PubMedQA abstracts that share no exact keyword overlap, while TF-IDF and BM25 tend to retrieve the lexically closest document—often the source document itself. RRF fusion ensures that documents highly ranked by the dense retriever survive the merging step even when absent from sparse-retriever top lists, resulting in a more diverse and genuinely relevant evidence set.

E. Discussion

Several cross-cutting observations merit discussion.

Bottleneck identification. Profiling reveals that LLM inference constitutes approximately 85% of per-query latency at the 500-query scale, with the remaining 15% split between retrieval (four concurrent retrievers) and RRF fusion. This partitioning explains why parallel efficiency degrades with scale: Amdahl’s Law predicts $S_{\max} = 1/(1-p) \approx 1/(0.15) \approx 6.7\times$ for the parallel fraction $p = 0.85$, but GPU contention in multi-threaded PyTorch inference further limits achieved speedup.

Accuracy context. The 51–53% achieved by our system benchmarks against a majority-class baseline (always predicting “yes”) of 56%. The small gap is attributable to two factors: (1) the model’s difficulty with negation in biomedical text; and (2) the absence of domain-specific fine-tuning. State-of-the-art biomedical models such as BioMedLM (2.7B parameters) achieve 78% on PubMedQA [1], but require substantially more compute and domain pre-training. Our system represents the practical operating point for fully local, no-API, commodity-GPU deployment.

Reproducibility. The entire system is reproducible from a single configuration file (`config.yaml`). All retriever indices, corpus embeddings, and LLM weights are cached after the first run. No external API calls are made at any stage.

VI. CONCLUSION

This paper presented a comprehensive serial-versus-parallel benchmark of a hybrid Retrieval-Augmented Generation system for biomedical question answering. By fusing sparse (TF-IDF word, TF-IDF character, BM25) and dense (sentence-transformer) retrieval via parameter-free Reciprocal Rank Fusion, and generating answers with a locally hosted Flan-T5-large model, the system achieves 53% accuracy on PubMedQA binary questions with 100% citation coverage across all tested configurations. Parallel execution via a two-level thread pool yields up to $1.37\times$ speedup over the serial baseline, with diminishing returns at higher query volumes and worker counts due to GPU contention during shared LLM inference. A prompt ablation study demonstrates that explicitly defining all three label semantics in the generation prompt reduces majority-class over-prediction by 16% and improves accuracy by approximately 3 percentage points.

Future directions include: (1) separating retrieval and generation onto distinct GPU devices to eliminate resource contention and approach linear speedup scaling; (2) fine-tuning the dense retriever encoder on PubMedQA training splits to improve domain-specific recall and hit rate; (3) replacing Flan-T5 with a parameter-efficient biomedical language model (e.g., a LoRA- adapted LLaMA or BioMedLM) to improve accuracy without sacrificing offline deployability; and (4) extending the benchmark to larger corpora (e.g., all of PubMed Central) to characterise retrieval latency scaling in the millions- of-documents regime.

ACKNOWLEDGMENT

The authors thank the open-source communities behind Hugging Face Transformers, sentence-transformers, FAISS, and scikit-learn for providing the foundational tools used in this work.

REFERENCES

- [1] S. Singhal, S. Azizi, T. Tu, S. S. Mahdavi, J. Wei, H. W. Chung, N. Scales, A. Tanwani, H. Cole-Lewis, S. Pföhl, *et al.*, “Towards expert-level medical question answering with large language models,” *arXiv preprint arXiv:2305.09617*, 2023.
- [2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [3] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, *et al.*, “Language models are few-shot learners,” in *NeurIPS*, vol. 33, pp. 1877–1901, 2020.
- [4] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-T. Yih, “Dense passage retrieval for open-domain question answering,” in *Proc. EMNLP*, pp. 6769–6781, 2020.
- [5] S. Shuster, S. Poff, M. Chen, D. Kiela, and J. Weston, “Retrieval augmentation reduces hallucination in conversation,” in *Findings of EMNLP*, pp. 3784–3803, 2021.
- [6] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Information Processing and Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [7] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [8] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-T. Yih, “Dense passage retrieval for open-domain question answering,” in *Proc. EMNLP*, pp. 6769–6781, 2020.
- [9] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. EMNLP*, pp. 3982–3992, 2019.
- [10] G. Izacard and E. Grave, “Leveraging passage retrieval with generative models for open domain question answering,” in *Proc. EACL*, pp. 874–880, 2021.
- [11] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal rank fusion outperforms Condorcet and individual rank learning methods,” in *Proc. ACM SIGIR*, pp. 758–759, 2009.
- [12] Q. Jin, B. Dhingra, Z. Liu, W. W. Cohen, and X. Lu, “PubMedQA: A biomedical research question answering dataset,” in *Proc. EMNLP*, pp. 2567–2577, 2019.
- [13] A. Ben Abacha and D. Demner-Fushman, “A question-entailment approach to question answering,” *BMC Bioinformatics*, vol. 20, no. 511, 2019.
- [14] D. Jin, E. Pan, N. Oufattole, W.-H. Weng, H. Fang, and P. Szolovits, “What disease does this patient have? A large-scale open domain question answering dataset from medical exams,” *Applied Sciences*, vol. 11, no. 14, p. 6421, 2021.
- [15] J. Lee, W. Yoon, S. Kim, D. Kim, S. Kim, C. H. So, and J. Kang, “BioBERT: A pre-trained biomedical language representation model for biomedical text mining,” *Bioinformatics*, vol. 36, no. 4, pp. 1234–1240, 2020.
- [16] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [17] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in *Proc. AFIPS Spring Joint Computer Conference*, pp. 483–485, 1967.
- [18] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [19] H. W. Chung, L. Hou, S. Longpre, B. Zoph, Y. Tay, W. Fedus, Y. Li, X. Wang, M. Dehghani, S. Brahma, A. Webson, S. S. Gu, Z. Dai, M. Suzgun, X. Chen, and Q. V. Le, “Scaling instruction-finetuned language models,” *Journal of Machine Learning Research*, vol. 25, no. 70, pp. 1–53, 2024.
- [20] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.
- [21] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. Le Scao, S. Gugger, M. Drame, Q. Lhoest, and A. M. Rush, “Transformers: State-of-the-art natural language processing,” in *Proc. EMNLP (System Demonstrations)*, pp. 38–45, 2020.